

Databases

Special topics datamodelering

Inhoud

- Recursieve relaties
- Tijdsafhankelijke attributen
- Overerving



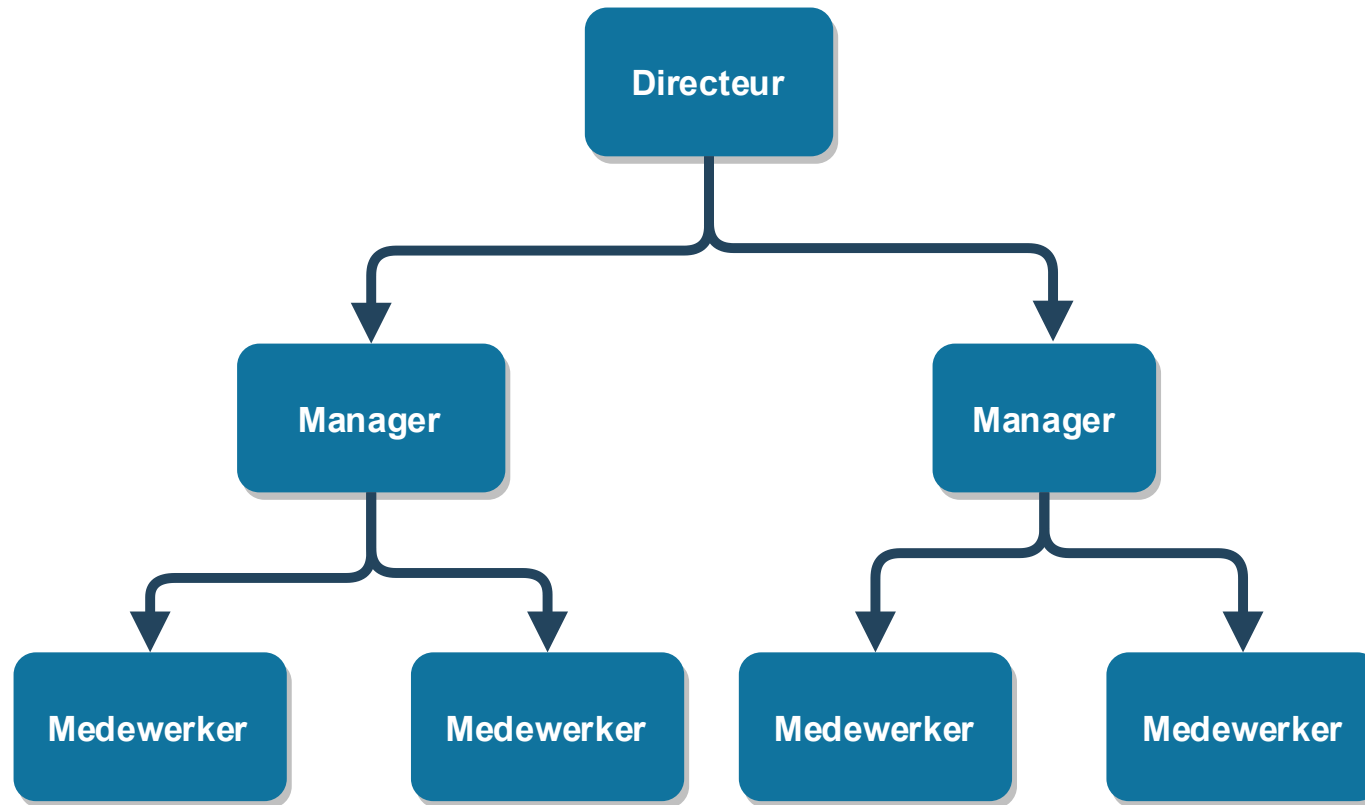
Special topics

Recuratieve relaties

Recursieve Relaties

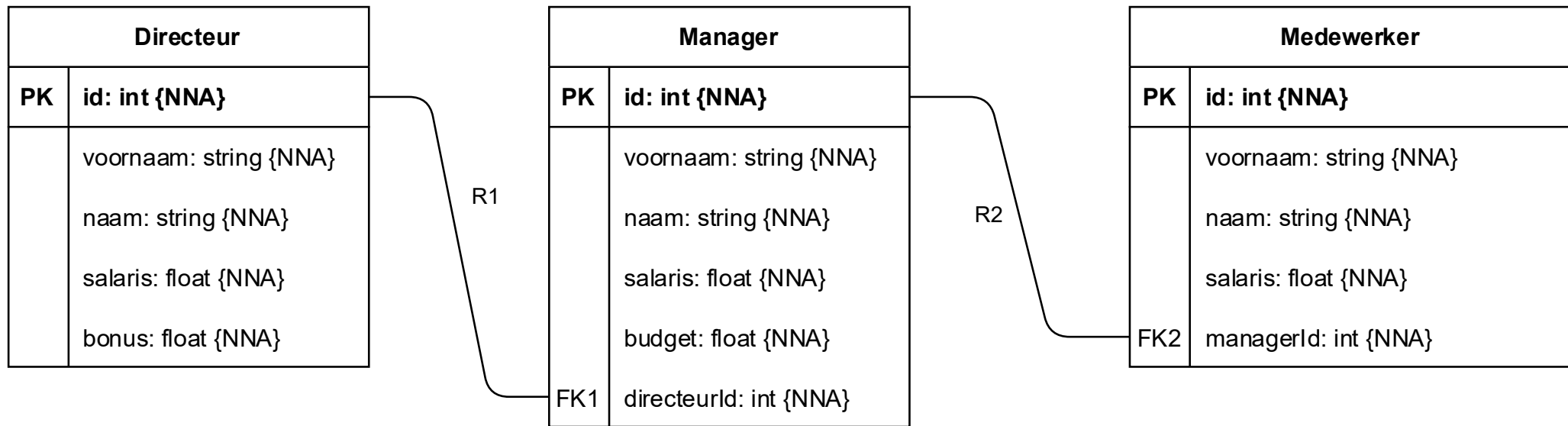
- Entiteiten die met zichzelf in relatie staan
- Recursieve relaties maken het modelleren van hiërarchische data mogelijk
- Veel voorkomend bij boomstructuren
 - Organigrammen
 - Categorisatie met sub-categorieën
 - Stambomen
 - Software pakketten en hun dependencies

Recursieve Relaties



Optie 1: Hiërarchisch model

- Een entiteit per niveau
- De hiërarchische structuur wordt volledig overgenomen in het model



Text is not SVG - cannot display

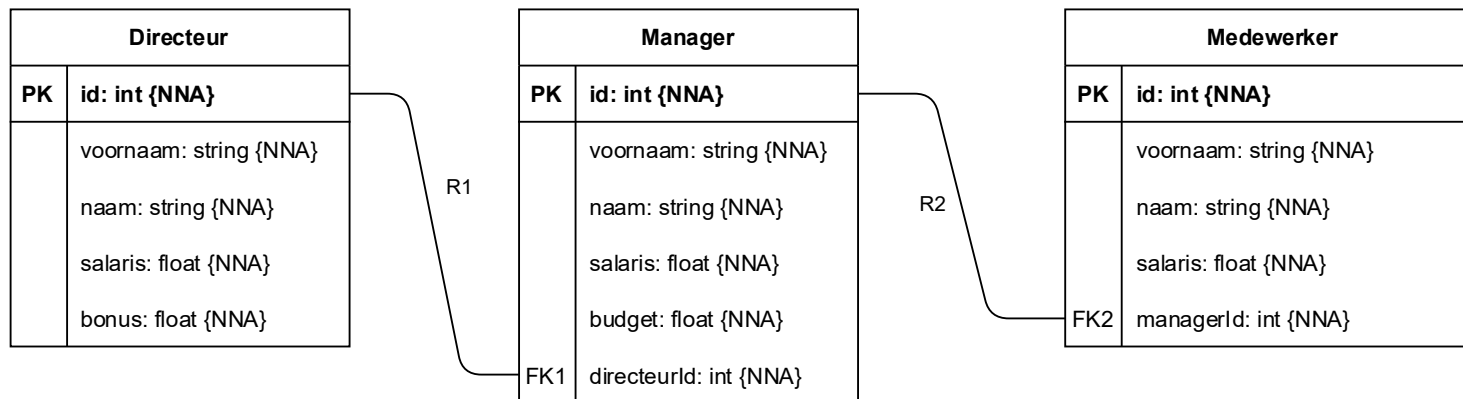
Optie 1: Hiërarchisch model

Voordelen

- Duidelijk model
- Veel flexibiliteit omtrent (verplichte) relaties en attributen

Nadelen

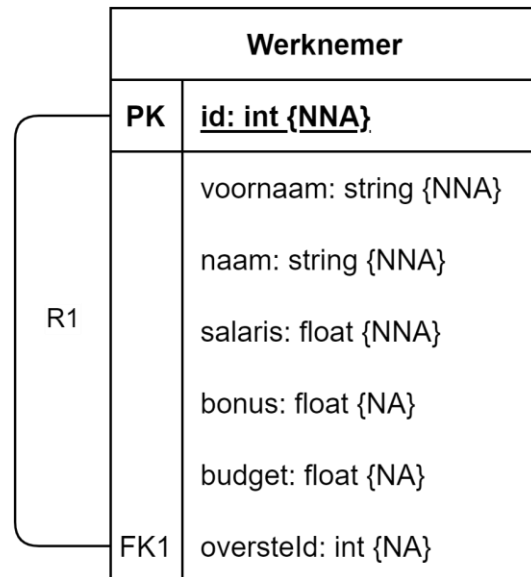
- Groter model
- Duplicatie van verschillende attributen



Text is not SVG - cannot display

Optie 2: Recursieve relatie

- De hiërarchische structuur is niet meer expliciet aanwezig
- Een algemene entiteit die een relatie met zichzelf heeft



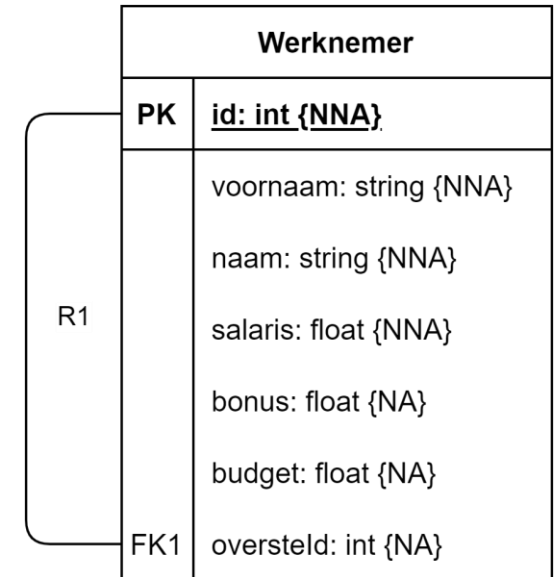
Optie 2: Recursieve relatie

Voordelen

- Kleiner model
- Attributen moeten niet in verschillende entiteiten gedefinieerd worden

Nadelen

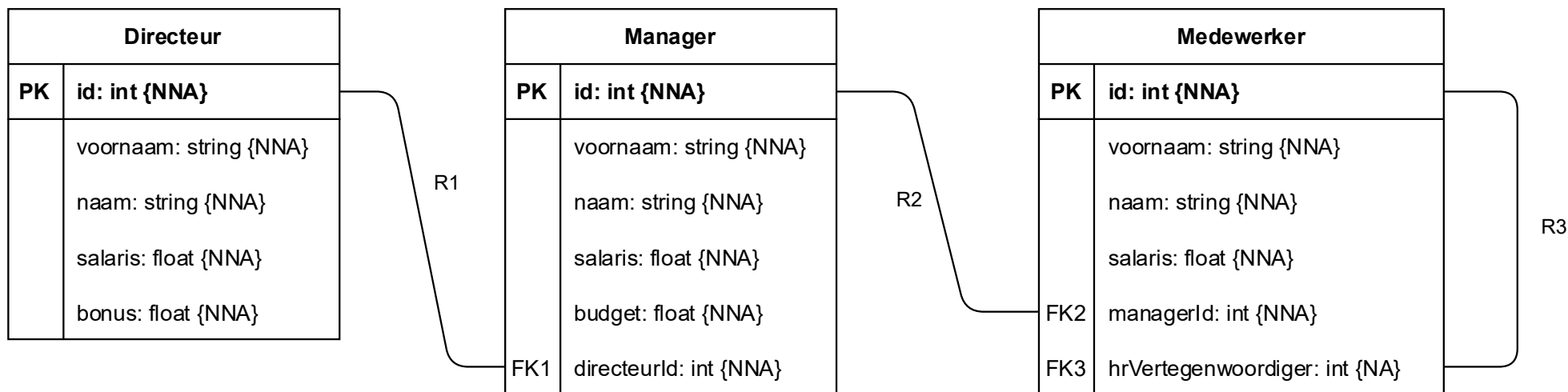
- Flexibiliteit is weg
 - *bonus* en *budget* waren NNA in de hiërarchische versie maar moeten nu NA worden



Rekursieve relaties

Stel: elke medewerker heeft een HR vertegenwoordiger die garandeert dat een werknemer rechtvaardig behandeld wordt door zijn oversten

- Hiërarchisch → nieuwe recursieve relatie tussen *Medewerker* en zichzelf

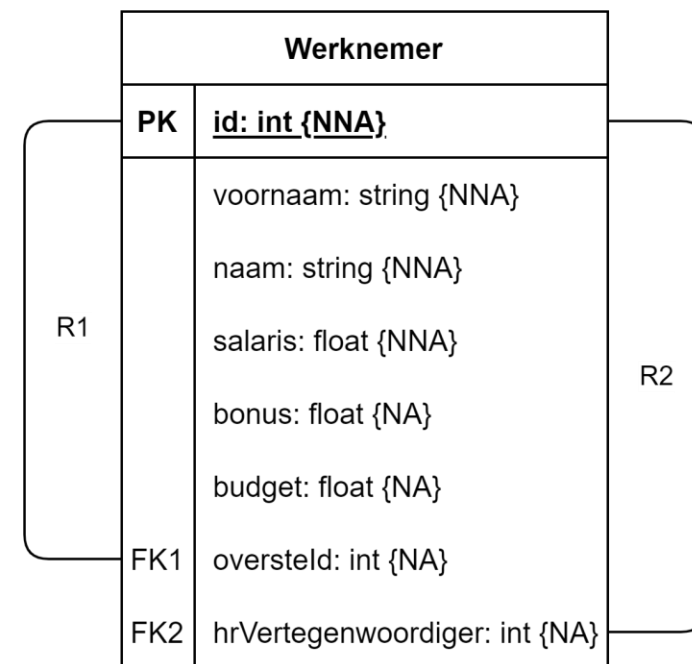


Text is not SVG - cannot display

Recursieve relaties

Stel: elke medewerker heeft een HR vertegenwoordiger die garandeert dat een werknemer rechtvaardig behandeld wordt door zijn oversten

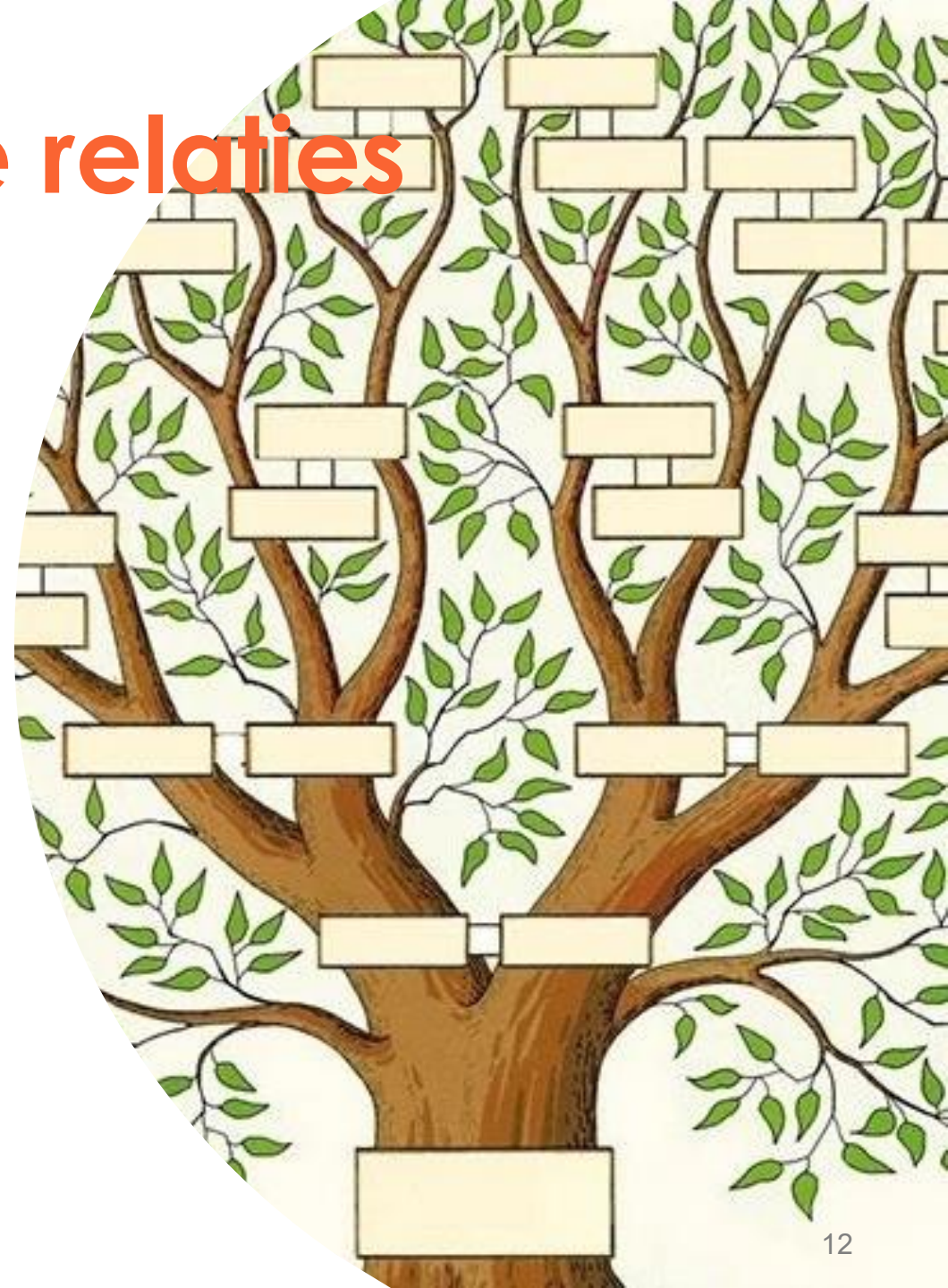
- Hiërarchisch → nieuwe recursieve relatie tussen *Medewerker* en zichzelf
- Recursief → nieuwe recursieve relatie tussen *Werknemer* en zichzelf
 - Minder duidelijk en de directeur kan nu ook een HR verantwoordelijke hebben, het model geeft dus de werkelijkheid minder goed weer



Meervoudige recursieve relaties

Beschouw een stamboom of vriend-relaties op een sociaal netwerk

- Een persoon kan geen of juist meerdere broers en zussen hebben
- Een persoon kan geen vrienden hebben of met honderden mensen “bevriend” zijn op een sociaal netwerk



Meervoudige recursieve relaties

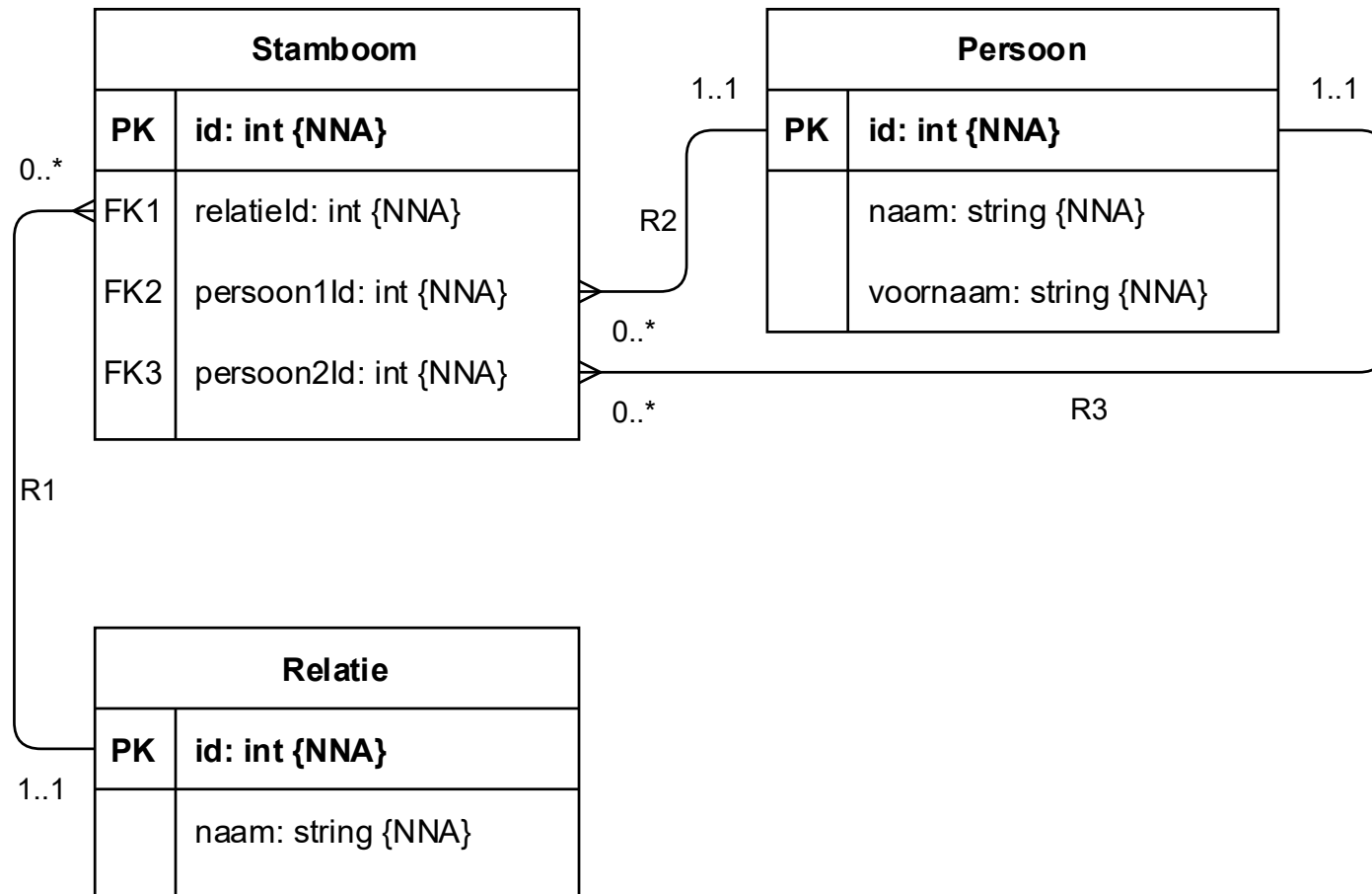
Beschouw een stamboom of vriend-relaties op een sociaal netwerk

- Een persoon kan geen of juist meerdere broers en zussen hebben
- Een persoon kan geen vrienden hebben of met honderden mensen “bevriend” zijn op een sociaal netwerk

Veel-op-veel relatie

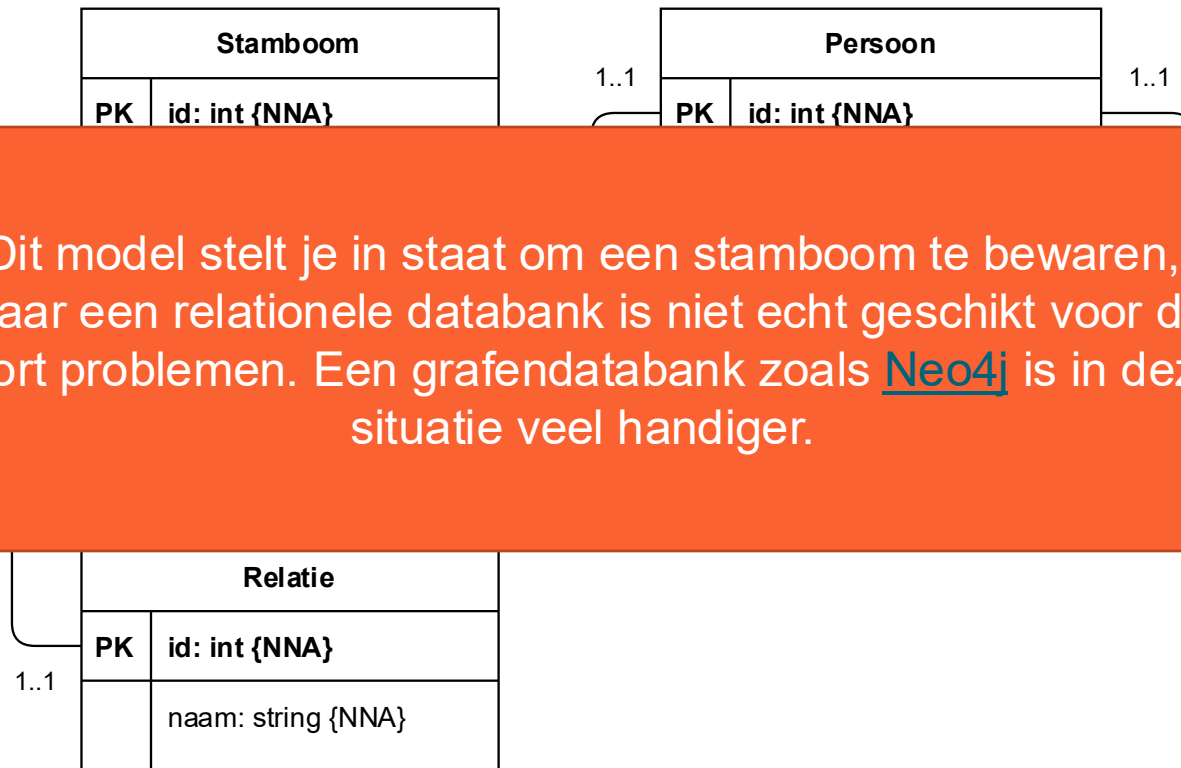
- Recursieve associatie entiteiten bieden hier een oplossing

Meervoudige recursieve relaties



Viewer does not support full SVG 1.1

Meervoudige recursieve relaties



Dit model stelt je in staat om een stamboom te bewaren, maar een relationele databank is niet echt geschikt voor dit soort problemen. Een grafendatabank zoals [Neo4j](#) is in deze situatie veel handiger.

Viewer does not support full SVG 1.1

Rekursieve relaties & Delete operations

- In recursieve relaties kunnen “in principe” verschillende keren een DTN/DTC gebruikt worden.
- Alhoewel dit logisch lijkt, ondersteund **SQL Server dit niet**, omdat dit tot lussen of meerdere cascade paths kan leiden.
- Stel dat we volgende rijen in een tabel hebben zitten (waar fk een foreign key naar dezelfde tabel is)

- Startwaardes (nog ok)

id	fk
1	NULL
2	1
3	2

- updaten eerste rij

id	fk
1	3
2	1
3	2

- rij 2 verwijderen



Recursieve relaties & Delete operations

- In recursieve relaties kunnen “in principe” verschillende keren een DTN/DTC gebruikt worden.
- Alhoewel dit logisch lijkt, ondersteund **SQL Server dit niet**, omdat dit tot lussen of meerdere cascade paths kan leiden.
- Stel dat we volgende rijen in een tabel hebben zitten (waar fk een foreign key naar dezelfde tabel is)

- Startwaardes (nog ok)

id	fk
1	NULL
2	1
3	2

- updaten eerste rij

id	fk
1	3
2	1
3	2

- rij 2 verwijderen

Er ontstaat nu een cascade (of set null lus). Stel dat we rij 2 verwijderen, dan wordt het cascade (of set null) path 2 - 3 -> 1 -> 2 -> 3 -> 1 -> 2 ->

...

Duidelijk een probleem en iets wat we niet zouden mogen aanleren.

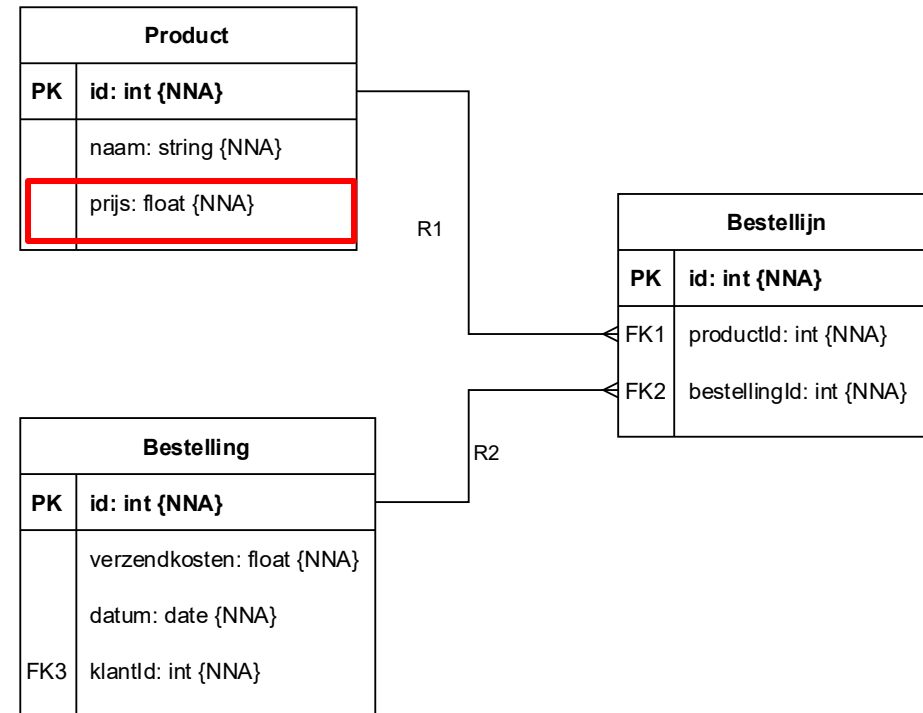
Special topics

Tijdsafhankelijke attributen

Tijdsafhankelijke attributen

Attributen kunnen doorheen de tijd van waarde wijzigen en historische gegevens moeten beschikbaar blijven

- Optie 1: Enkel de actuele waarde

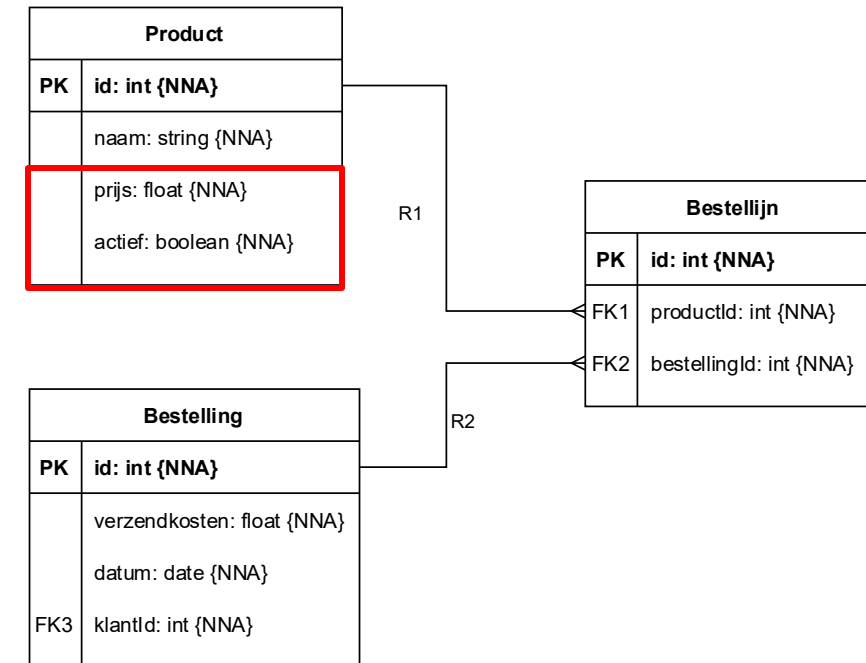


Viewer does not support full SVG 1.1

Tijdsafhankelijke attributen

Attributen kunnen doorheen de tijd van waarde wijzigen en historische gegevens moeten beschikbaar blijven

- Optie 1: Enkel de actuele waarde
- Optie 2: Actuele waarde en de historische waarden
 - Wijzigingen in prijs zijn **zeer uitzonderlijk** → booleaans attribuut toevoegen, een prijswijziging wordt beschouwd als een nieuw product

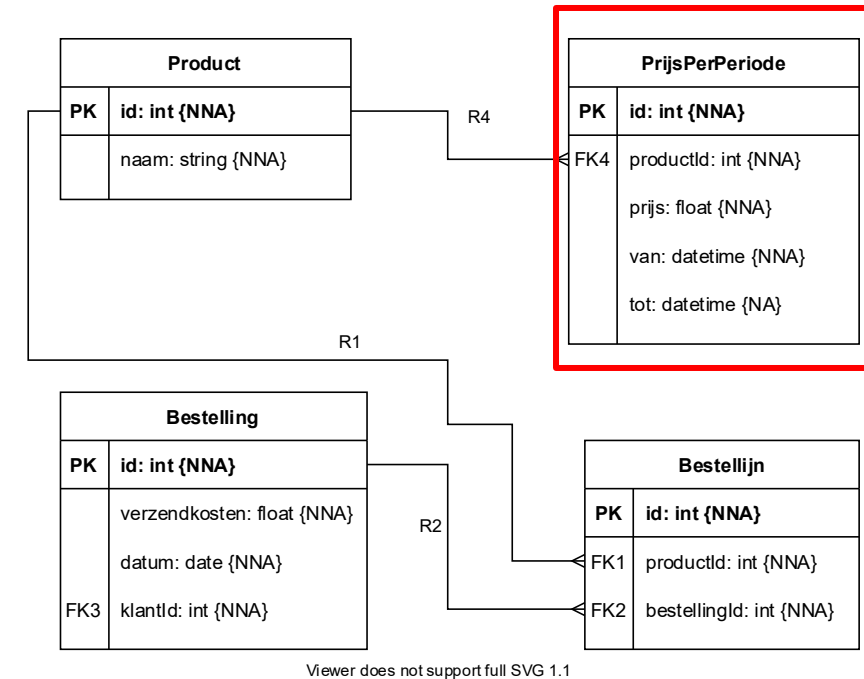


Viewer does not support full SVG 1.1

Tijdsafhankelijke attributen

Attributen kunnen doorheen de tijd van waarde wijzigen en historische gegevens moeten beschikbaar blijven

- Optie 1: Enkel de actuele waarde
- Optie 2: Actuele waarde en de historische waarden
 - Wijzigingen in prijs zijn zeer uitzonderlijk → booleaans attribuut toevoegen, een prijswijziging wordt beschouwd als een nieuw product
 - **Frequente wijzigingen** → Extra karakteristieke entiteit met historische waarden



Special topics

Overerving

Overerving

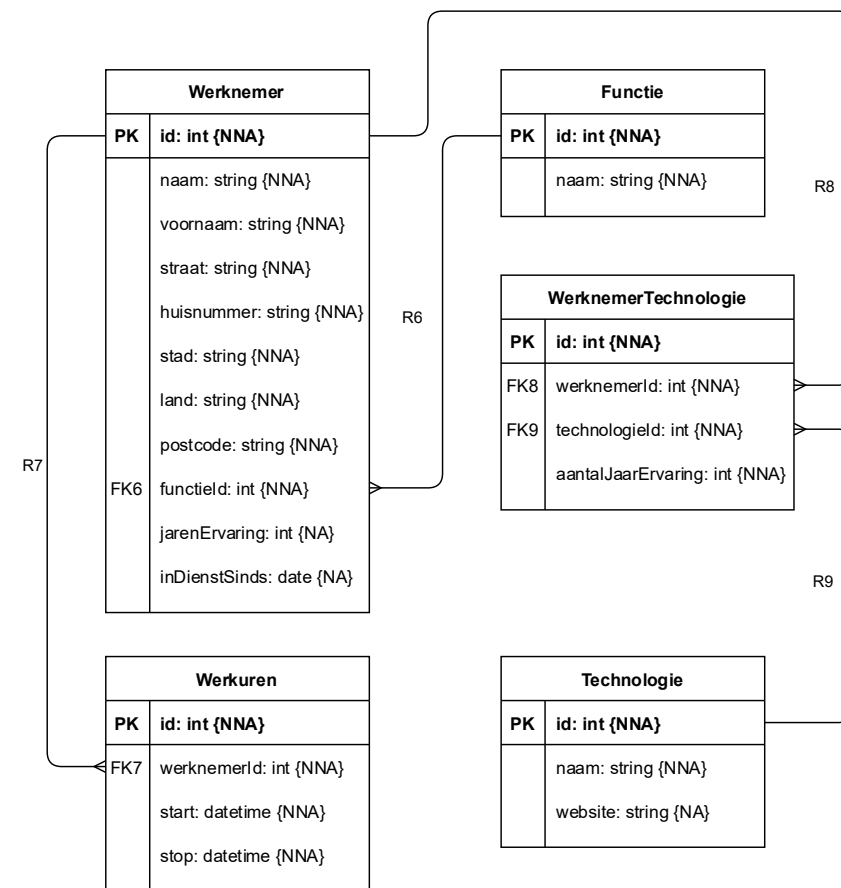
We bouwen een HR beheer applicatie voor een webshop.

- Voor deze webshop werken magazijniers die de producten inpakken en verzenden en technisch personeel die de technische aspecten van de website up-to-date houden.
- Van deze werknemers zijn NAW gegevens vereist.
- Verder moeten we van magazijniers weten hoeveel uren ze gewerkt hebben en wanneer ze in dienst getreden zijn.
- Tenslotte moeten we van de technische werknemers weten hoeveel jaar ervaring ze hebben in de sector, en of ze in een junior, medior, senior of lead positie werken en met welke technologieën ze bekend zijn.

Overerving

Optie 1: Eén entiteit

- Niet gemeenschappelijke attributen moeten NA zijn
- Discriminator attribuut om onderscheid te maken
- Voordelen
 - ?

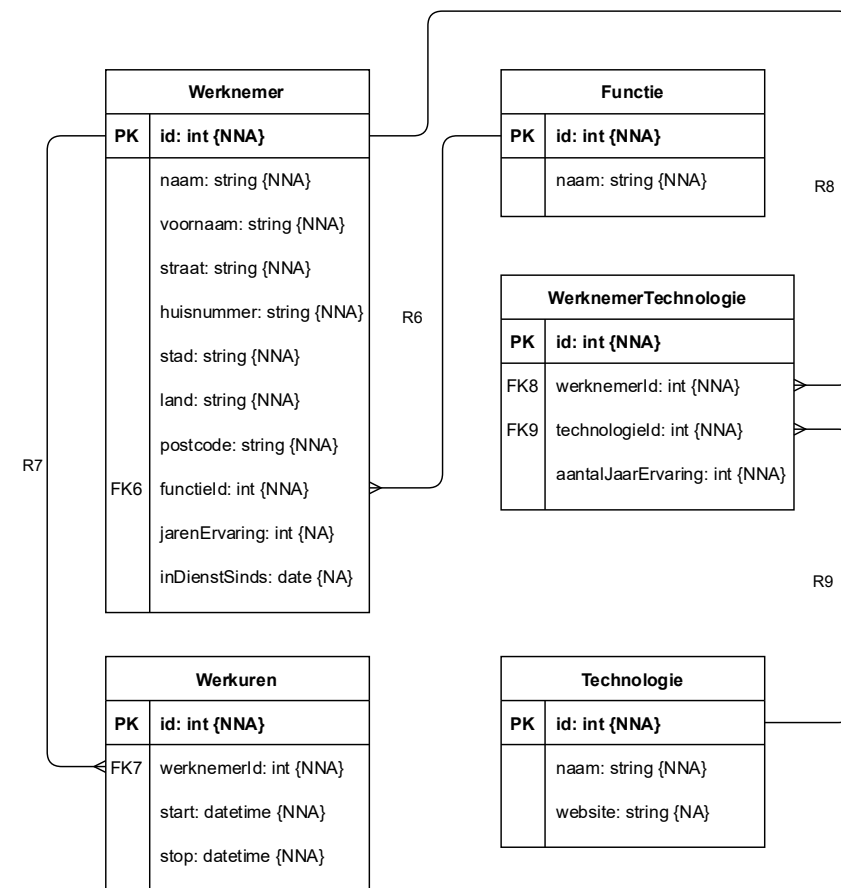


Viewer does not support full SVG 1.1

Overerving

Optie 1: Eén entiteit

- Niet gemeenschappelijke attributen moeten NA zijn
- Discriminator attribuut om onderscheid te maken
- Voordelen
 - Geen SQL joins nodig
 - Wijziging → slechts 1 INSERT/UPDATE
 - Geen redundante attribuutdefinities
- Nadelen
 - ?

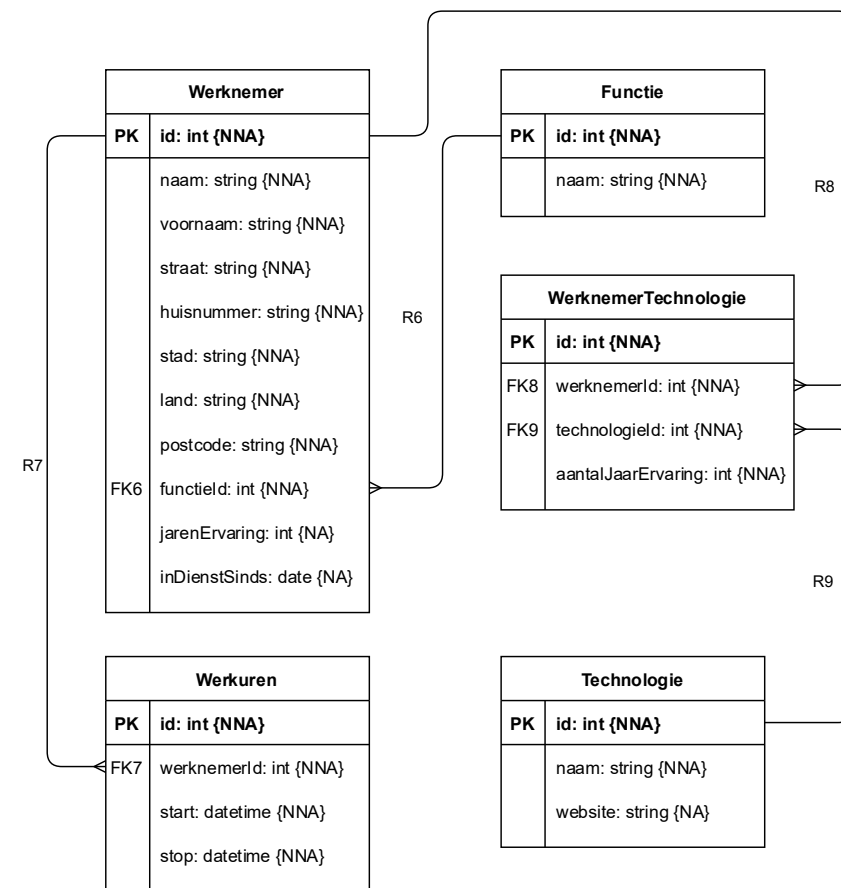


Viewer does not support full SVG 1.1

Overerving

Optie 1: Eén entiteit

- Niet gemeenschappelijke attributen moeten NA zijn
- Discriminator attribuut om onderscheid te maken
- Voordelen
 - Geen SQL joins nodig
 - Wijziging → slechts 1 INSERT/UPDATE
 - Geen redundante attribuutdefinities
- Nadelen
 - De tabellen kunnen groot worden
 - Lege attributen
 - Bedrijfsregels niet geïmplementeerd



Viewer does not support full SVG 1.1

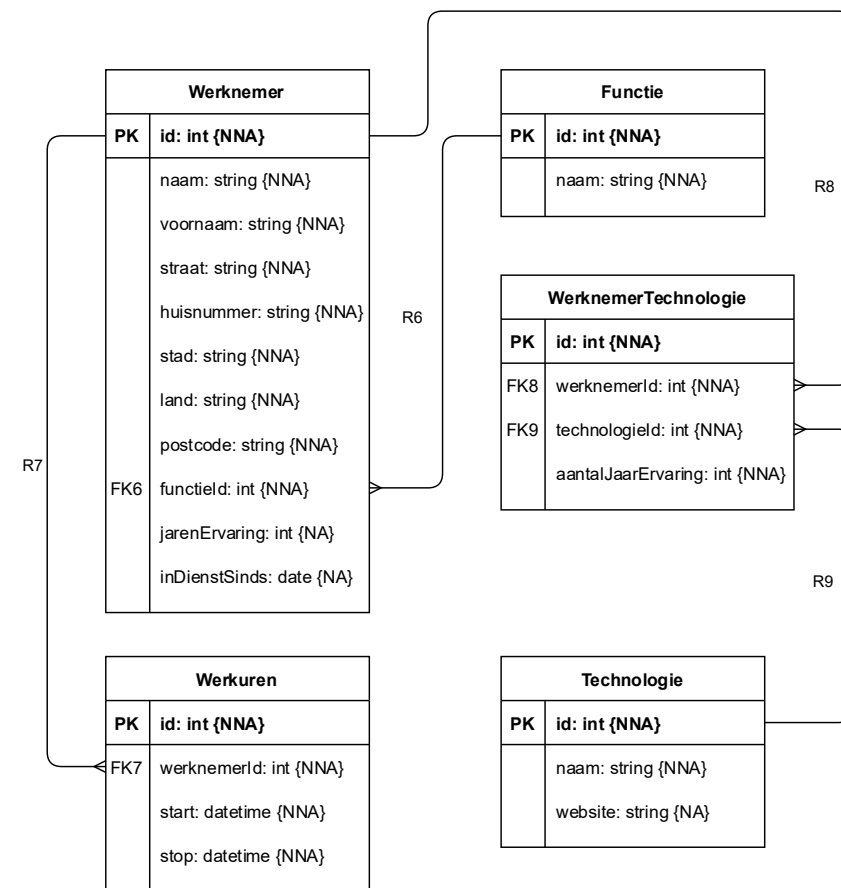
Overerving

Optie 1: Eén entiteit

- Niet gemeenschappelijke attributen moeten NA zijn

Single tabel is een goede keuze indien subklassen (technisch medewerker en magazijnier) weinig unieke attributen bevatten en het onwaarschijnlijk is dat er nog klassen toegevoegd en/of aangepast zullen worden.

- Nadelen
 - De tabellen kunnen groot worden
 - Lege attributen
 - Bedrijfsregels niet geïmplementeerd

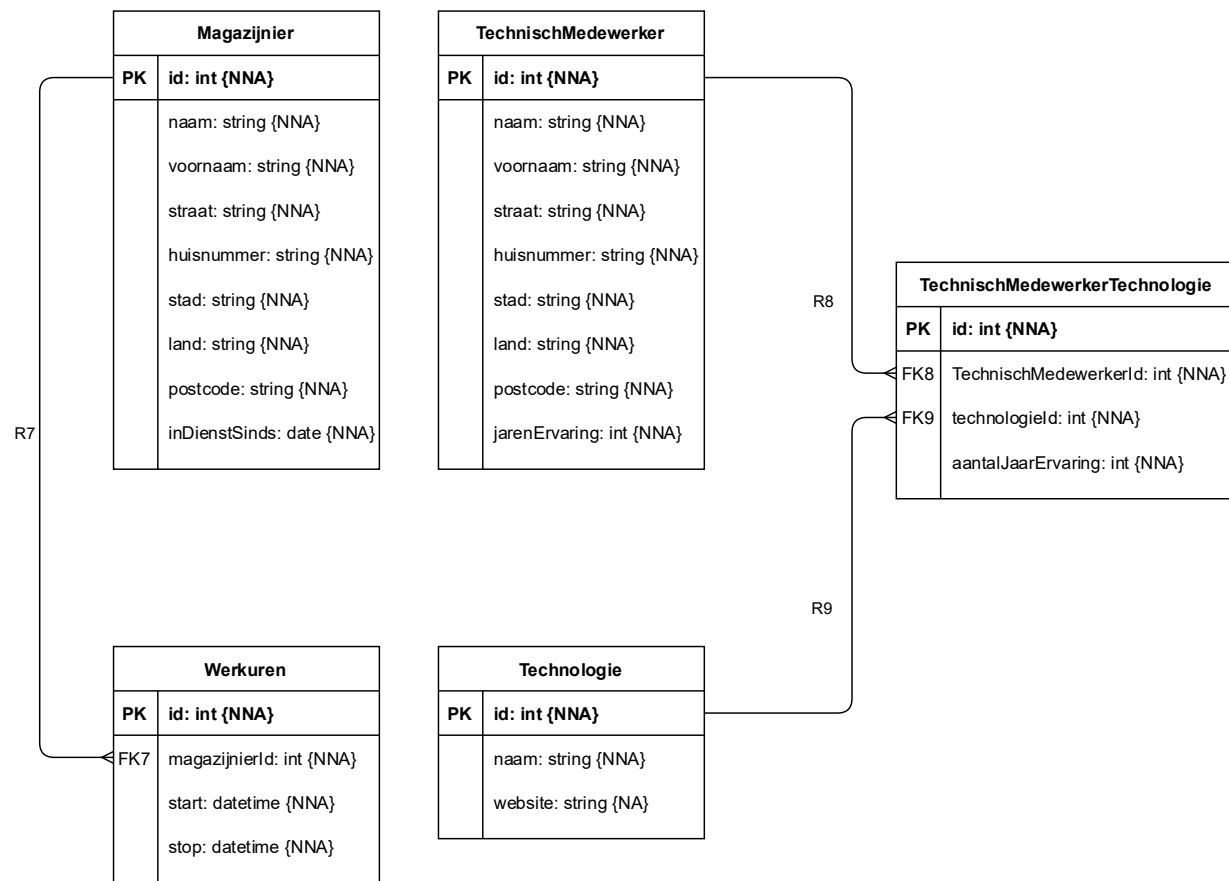


Viewer does not support full SVG 1.1

Overerving

Optie 2: Entiteit per klasse

- Duidelijker model
- Geen NA/discriminator attribuut nodig
- Voordelen
 - ?

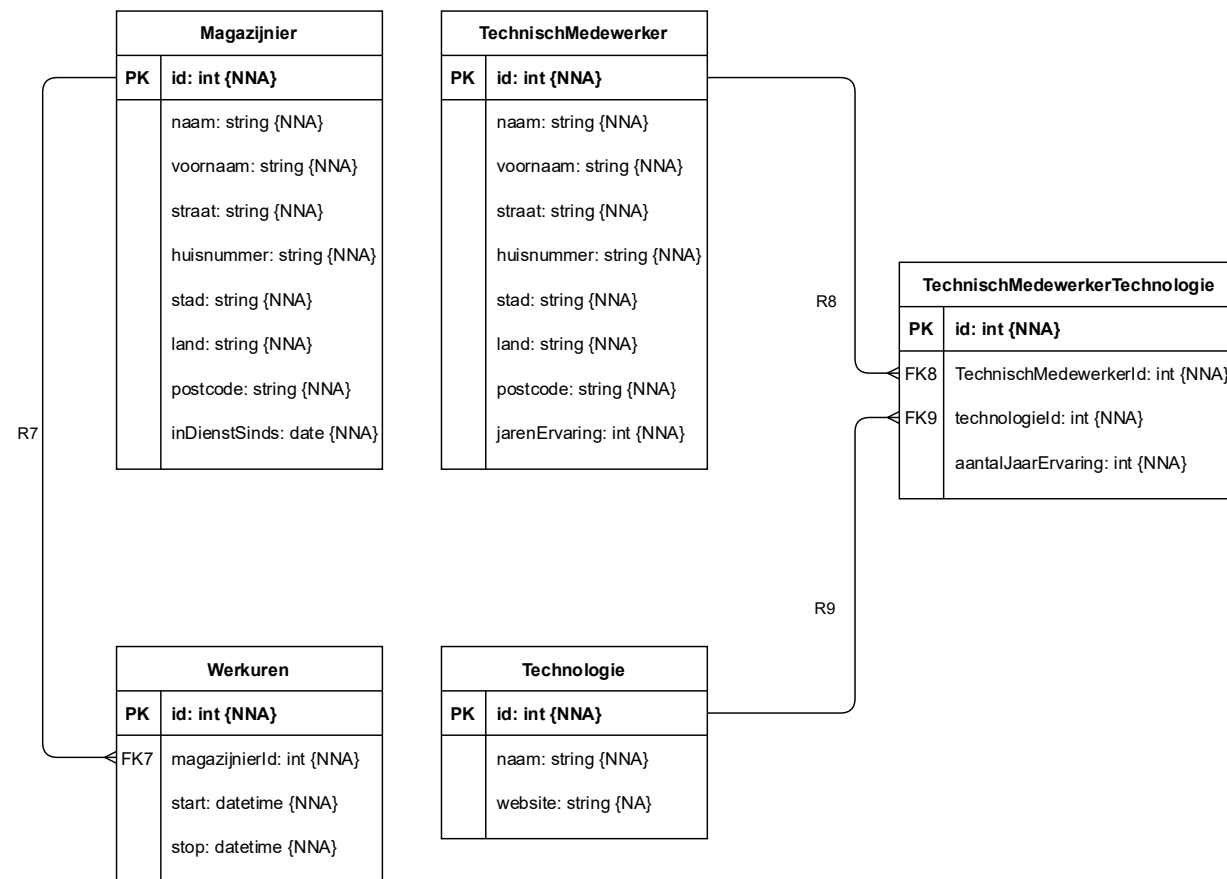


Viewer does not support full SVG 1.1

Overerving

Optie 2: Entiteit per klasse

- Duidelijker model
- Geen NA/discriminator attribuut nodig
- Voordelen
 - Subklassen eenvoudig toe te voegen zonder de bestaande entiteiten aan te passen
 - Geen SQL join nodig
 - Wijziging → slechts 1 INSERT/UPDATE
 - Bedrijfsregels worden geïmplementeerd
- Nadelen
 - ?

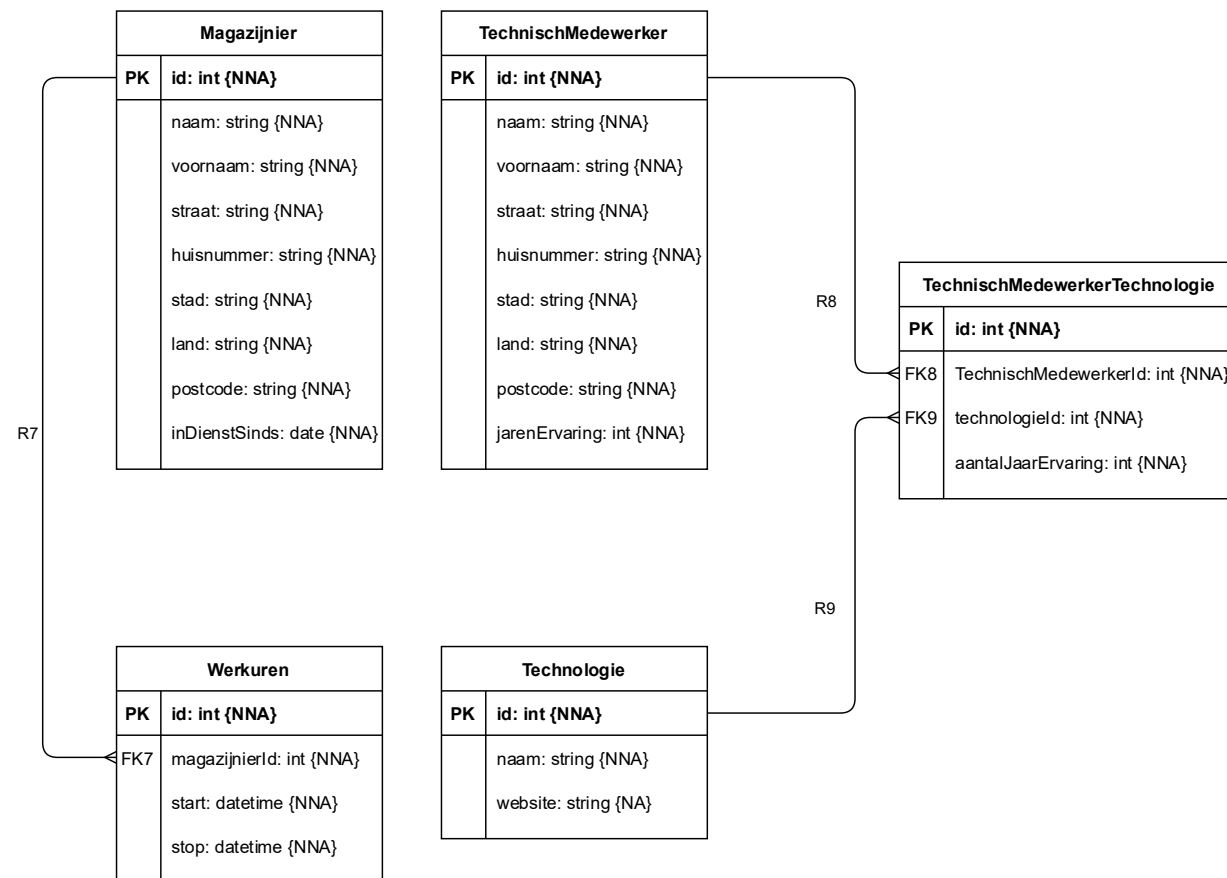


Viewer does not support full SVG 1.1

Overerving

Optie 2: Entiteit per klasse

- Duidelijker model
- Geen NA/discriminator attribuut nodig
- Voordelen
 - Subklassen eenvoudig toe te voegen zonder de bestaande entiteiten aan te passen
 - Geen SQL join nodig
 - Wijziging → slechts 1 INSERT/UPDATE
 - Bedrijfsregels worden geïmplementeerd
- Nadelen
 - Veel redundante attribuutdefinities
 - Veel tabellen
 - Alle werknemers opvragen → UNION



Viewer does not support full SVG 1.1

Overerving

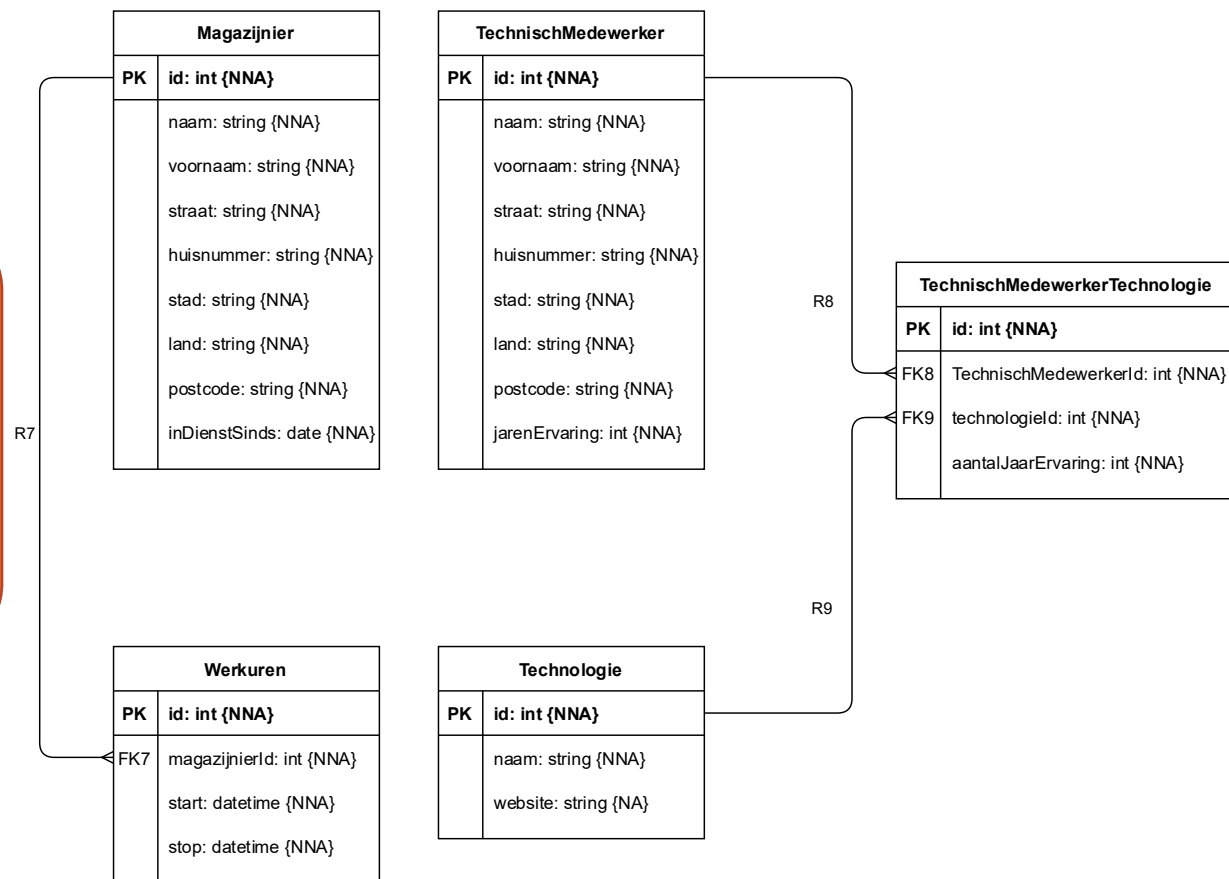
Optie 2: Entiteit per klasse

- Duidelijker model

Dit is een goede keuze indien het waarschijnlijk is dat er nog veel wijzigingen doorgevoerd worden en er meer unieke attributen dan overgeërfd attributen zijn. De subtypes moeten disjunct zijn, i.e. een magazijnier kan niet ook een technisch medewerker zijn.

Wijziging 7: Inserts & UPDATE

- Nadelen
 - Veel dubbele attributen
 - Veel tabellen
 - Alle werknemers opvragen → UNION

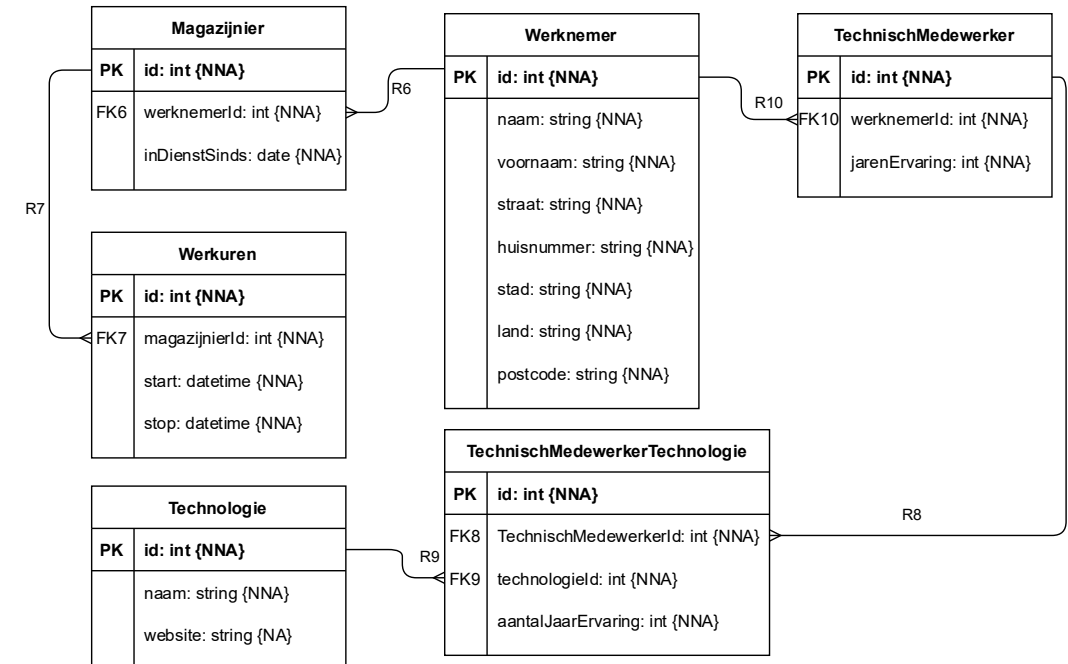


Viewer does not support full SVG 1.1

Overerving

Optie 3: Joined tables

- Gemeenschappelijke entiteit (supertype) met de gemeenschappelijke attributen
- Per subtype een entiteit met unieke attributen
- Voordelen
 - ?

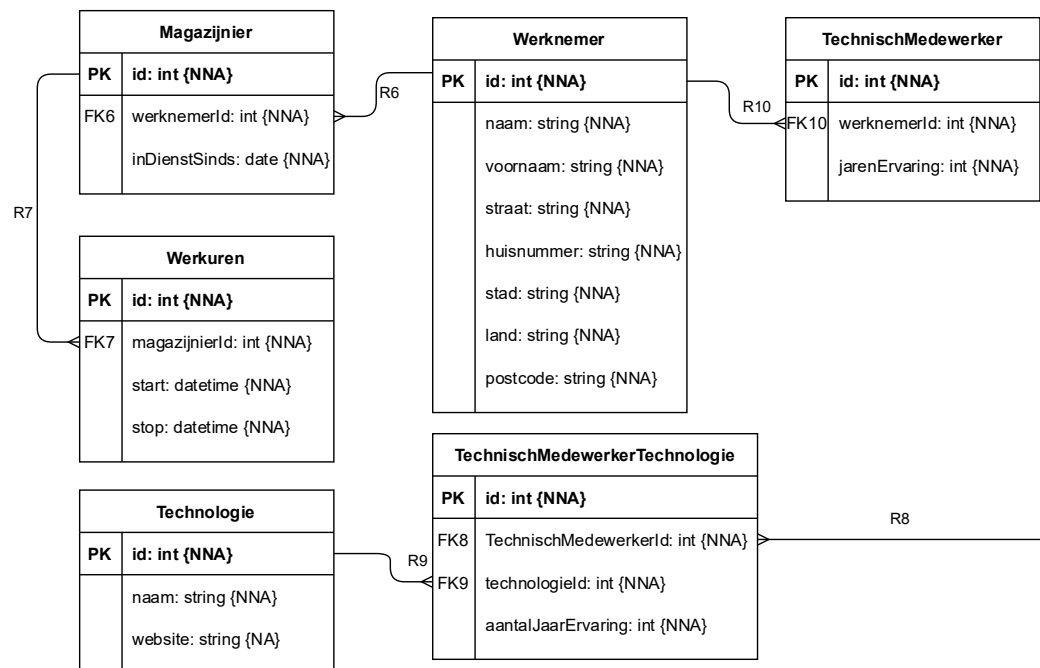


Viewer does not support full SVG 1.1

Overerving

Optie 3: Joined tables

- Gemeenschappelijke entiteit (supertype) met de gemeenschappelijke attributen
- Per subtype een entiteit met unieke attributen
- Voordelen
 - Subklassen eenvoudig toe te voegen zonder de bestaande entiteiten aan te passen
 - Geen dubbele attributen
- Nadelen
 - ?

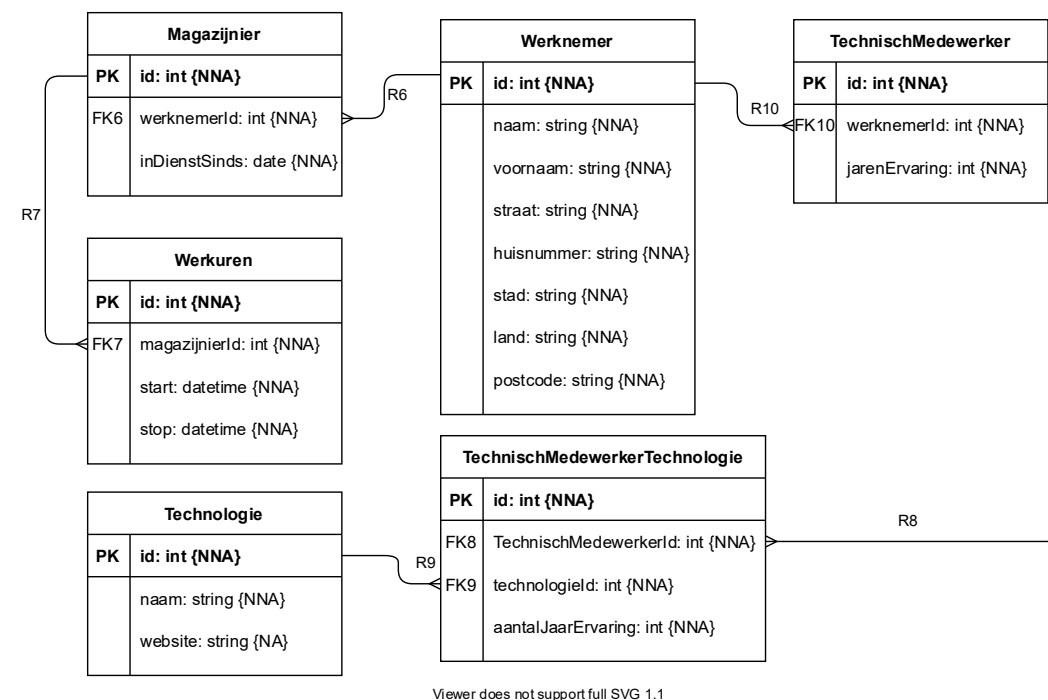


Viewer does not support full SVG 1.1

Overerving

Optie 3: Joined tables

- Gemeenschappelijke entiteit (supertype) met de gemeenschappelijke attributen
- Per subtype een entiteit met unieke attributen
- Voordelen
 - Subklassen eenvoudig toe te voegen zonder de bestaande entiteiten aan te passen
 - Geen dubbele attributen
- Nadelen
 - SQL JOINS nodig
 - Wijziging → meerdere INSERT/UPDATE opdrachten
 - Indien de overervingsstructuur groot is, kan dit minder performant worden



Viewer does not support full SVG 1.1

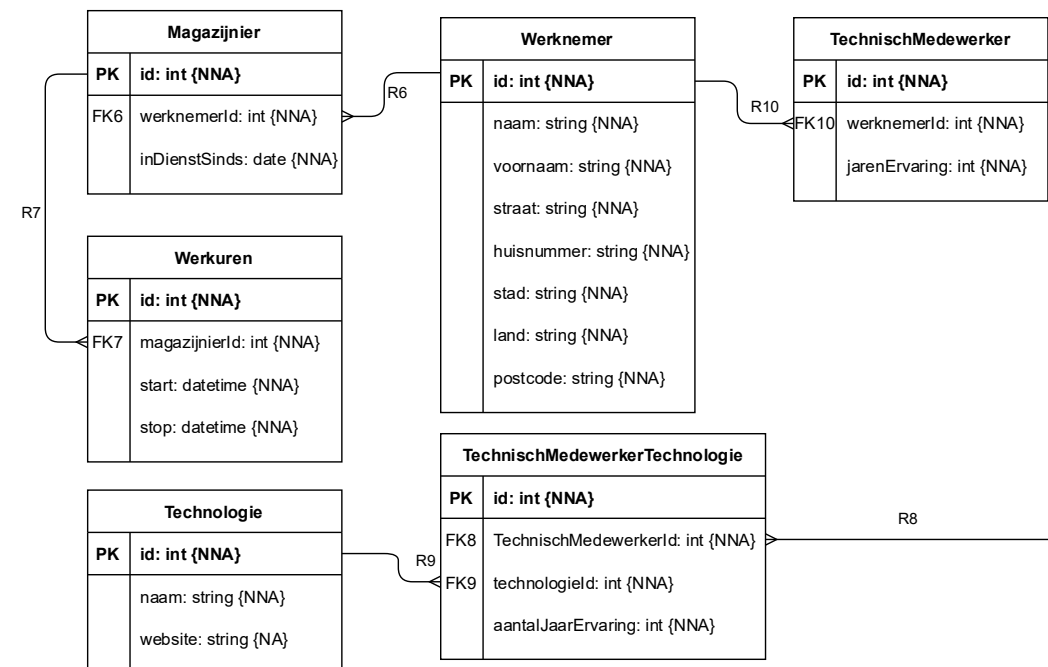
Overerving

Optie 3: Joined tables

- Gemeenschappelijke entiteit (supertype) met de

Dit is een goede keuze indien de subtypes veel unieke attributen hebben en het erg waarschijnlijk is dat er nog veel klassen toegevoegd of aangepast worden.

- Nadelen
 - SQL JOINS nodig
 - Wijziging → meerdere INSERT/UPDATE opdrachten



Viewer does not support full SVG 1.1